

DevSecOps Question Index - All 100 Questions

Q#	Question
Q1	What is DevSecOps? Shifting security left in the SDLC
Q2	CIA triad - Confidentiality, Integrity, Availability
Q3	Authentication vs Authorization - OAuth 2.0 flows, OIDC, SAML
Q4	TLS/SSL - certificates, handshake, mTLS, certificate chains
Q5	Encryption - symmetric, asymmetric, hashing, PKI
Q6	OWASP Top 10 - SQLi, XSS, CSRF, SSRF, broken auth
Q7	Secret management - Vault, Secrets Manager, sealed secrets
Q8	SSH security - key types, rotation, bastion hosts, certificates
Q9	Firewall and network security - WAF, IDS/IPS, DMZ, zero trust
Q10	Vulnerability scanning - CVEs, CVSS scores, NVD, patching
Q11	Dependency scanning - SCA, Dependabot, Renovate, Snyk
Q12	Container image scanning - Trivy, Gype, Docker Scout
Q13	Git security - signed commits, branch protection, git-secrets
Q14	Least privilege - IAM, RBAC, service accounts
Q15	Logging and audit trails - who did what, when, where
Q16	Multi-factor authentication - TOTP, hardware keys, push
Q17	Password policies and credential hygiene
Q18	Network segmentation and DNS security
Q19	Compliance - SOC 2, ISO 27001, PCI DSS, HIPAA, GDPR
Q20	Security in CI/CD - where to add security gates
Q21	SAST - static security testing, tools, false positives
Q22	DAST - dynamic testing, OWASP ZAP, authenticated scanning

Q23	SCA and SBOM - software composition, license compliance
Q24	IaC security - tfsec, checkov, kics, policy as code
Q25	JWT security - claims, signing, expiration, common attacks
Q26	Container security hardening - rootless, read-only, capabilities
Q27	Kubernetes security - RBAC, PSA, NetworkPolicies, secrets
Q28	Image signing - cosign, Notary, admission webhooks
Q29	Supply chain security - SLSA, provenance, Sigstore
Q30	Secret rotation and dynamic secrets - Vault credentials
Q31	CORS deep dive - preflight, origins, credentials, misconfigs
Q32	API security - rate limiting, input validation, OAuth vs keys
Q33	CSP and security headers - browser security mechanisms
Q34	Key management - HSMs, rotation, envelope encryption
Q35	Cloud security posture - CSPM, misconfiguration detection
Q36	Penetration testing - types, scope, tools, disclosure
Q37	Threat modeling - STRIDE, DREAD, attack trees
Q38	Secrets sprawl detection - repos, env, configs, wikis
Q39	Security monitoring - SIEM, SOC, detection rules
Q40	Incident response plan - preparation, containment, eradication
Q41	Certificate management at scale - cert-manager, ACME, CAs
Q42	K8s network policies deep dive - default deny, egress, DNS
Q43	Secrets in Kubernetes - ESO, sealed secrets, Vault agent
Q44	Docker security - daemon, socket, Bench for Security
Q45	IAM hardening - cross-account, boundaries, SCPs
Q46	Cloud security tools - GuardDuty, Security Hub, SCC, Defender

Q47	WAF tuning - rules, false positive management
Q48	Vulnerability prioritization - EPSS, reachability analysis
Q49	Security benchmarks - CIS for OS, Docker, K8s, cloud
Q50	API gateway security - JWT validation, mTLS, rate limiting
Q51	Zero trust architecture - never trust, always verify
Q52	mTLS everywhere - service mesh, SPIFFE/SPIRE, rotation
Q53	Policy as code - OPA/Gatekeeper, Kyverno, Sentinel
Q54	Runtime security - Falco, Sysdig, eBPF detection
Q55	Secrets management architecture - Vault HA, auto-unseal
Q56	RBAC vs ABAC vs ReBAC - modern access control models
Q57	Security architecture review - threat modeling microservices
Q58	Compliance as code - automated auditing, continuous compliance
Q59	Data protection - classification, DLP, tokenization, masking
Q60	Shift-right security - RASP, canary tokens, honeypots
Q61	Identity federation - OIDC, SAML, cross-cloud identity
Q62	Binary authorization - admission controllers, policy chains
Q63	Privileged access management - PAM, break-glass, JIT access
Q64	Serverless security - function permissions, event injection
Q65	Container forensics - evidence capture, analysis
Q66	Chaos security testing - security Game Days, purple team
Q67	Red team vs blue team - offensive vs defensive security
Q68	Container runtime isolation - gVisor, Kata, Firecracker
Q69	Bug bounty programs - scope, rewards, disclosure
Q70	Security culture - champions, training, gamification

Q71	AWS credentials leaked on GitHub - incident response
Q72	Container escape - Pod reaching host
Q73	Critical CVE in production dependency - patch or mitigate
Q74	K8s RBAC misconfigured - Pod has cluster-admin
Q75	SQL injection in production API
Q76	S3 bucket with PII exposed publicly
Q77	TLS certificate expired - production service down
Q78	Insider threat - employee exfiltrating data
Q79	Ransomware encrypted production servers
Q80	Supply chain attack - compromised npm package
Q81	DDoS attack overwhelming application
Q82	Secrets in Docker image layers - detection and cleanup
Q83	SSRF accessing cloud metadata service
Q84	K8s admission webhook bypassed - unsigned images
Q85	Log4Shell (CVE-2021-44228) - detect, patch, verify
Q86	CI/CD pipeline compromised - malicious code injection
Q87	XSS in customer-facing application
Q88	Database credentials hardcoded in source code
Q89	Security audit - 500+ critical findings prioritization
Q90	Design a complete DevSecOps pipeline
Q91	JWT token stolen - session hijacking and revocation
Q92	K8s ServiceAccount token auto-mounted giving API access
Q93	Cryptojacking - EC2 mining crypto undetected
Q94	Dependency confusion - internal package hijacked

Q95	mTLS certificate expired - service mesh down
Q96	WAF blocking legitimate traffic - revenue loss
Q97	Terraform state with secrets exposed in S3
Q98	Developer laptop stolen with production access
Q99	DNS hijacking - traffic redirected to attacker
Q100	Privileged container in production - runtime detection

SECTION 1 - Junior Security Fundamentals (Q1-Q20)

Q1 What is DevSecOps? Shifting security left

The Simple Answer

DevSecOps integrates security into every phase of the software development lifecycle rather than treating it as a final gate before production. "Shifting left" means finding and fixing security issues early (in development and CI) when they are cheap to fix, rather than late (in production) when they are expensive and dangerous.

The Analogy - Building a House

Traditional security is like hiring a building inspector only after the house is fully built. If they find the foundation is wrong, you demolish and rebuild - expensive and slow. DevSecOps is like having the inspector review blueprints, check the foundation during construction, inspect wiring as it is installed, and verify the final product. Issues caught early cost 10x less to fix.

What DevSecOps Looks Like in Practice

Plan - Threat modeling during design. Security requirements in user stories. **Code** - IDE plugins for security linting. Pre-commit hooks for secret detection. Secure coding guidelines. **Build** - SAST scanning in CI. Dependency scanning (SCA). Container image scanning. **Test** - DAST against staging. Penetration testing. Security-focused integration tests. **Deploy** - Image signing verification. Admission webhooks blocking non-compliant resources. IaC security scanning. **Operate** - Runtime threat detection (Falco). Security monitoring (SIEM). Vulnerability management. Incident response. **Monitor** - Security dashboards. Compliance drift detection. Audit logging.

The cultural shift matters most. DevSecOps is not just tools - it is making every developer responsible for security, providing them with automated tools that catch issues without slowing them down, and creating a culture where security is everyone's job, not just the security team's.

Q2 CIA triad - Confidentiality, Integrity, Availability

The Simple Answer

The CIA triad is the foundation of all information security. Every security decision balances these three principles.

Confidentiality - Only authorized people can access data. Encryption, access control, classification. A breach of confidentiality: customer data leaked publicly.

Integrity - Data is accurate and untampered. Checksums, digital signatures, audit logs. A breach of integrity: an attacker modifies financial records.

Availability - Systems are accessible when needed. Redundancy, backups, DDoS protection. A breach of availability: ransomware makes systems unusable.

Real-World Trade-offs

More encryption (confidentiality) adds latency (reduces availability). Strict access control (confidentiality) slows development (reduces availability). Multiple replicas (availability) create more targets to protect (challenges confidentiality). Security architecture is about finding the right balance for your threat model.

Q3 Authentication vs Authorization - OAuth 2.0, OIDC, SAML

The Simple Answer

Authentication (AuthN) - Verifying WHO you are. Prove your identity with: something you know (password), something you have (phone, key), something you are (biometric).

Authorization (AuthZ) - Verifying WHAT you can do. After proving your identity, the system checks your permissions. Can you read this file? Can you deploy to production? Can you access this API?

OAuth 2.0 - Authorization Framework

OAuth 2.0 grants limited access to resources without sharing credentials. Key flows: **Authorization Code** - For web apps. User redirects to auth server, approves access, receives a code exchanged for tokens. Most secure for user-facing apps. **Client Credentials** - For service-to-service. No user involved. The service authenticates directly and receives a token. Used for backend API calls. **Authorization Code + PKCE** - For mobile and SPA apps. Adds a code verifier to prevent authorization code interception.

OIDC (OpenID Connect)

An identity layer on top of OAuth 2.0. OAuth gives you authorization (access tokens). OIDC adds authentication (ID tokens with user identity claims). Use OIDC when you need to know WHO the user is, not just grant access.

SAML

XML-based federation protocol. Common in enterprise environments. User authenticates with an Identity Provider (Okta, Azure AD), receives a SAML assertion, presents it to the Service Provider. Being replaced by OIDC for new implementations but still dominant in enterprise.

Q4 TLS/SSL - certificates, handshake, mTLS

The Simple Answer

TLS (Transport Layer Security) encrypts communication between client and server. SSL is the deprecated predecessor - when people say SSL, they usually mean TLS. HTTPS is HTTP over TLS.

The TLS Handshake (Simplified)

1. Client sends "Hello" with supported cipher suites.
2. Server responds with its certificate (containing the public key).
3. Client verifies the certificate against trusted CAs.
4. Client and server negotiate a symmetric session key.
5. All subsequent traffic is encrypted with the session key.

Certificate Chain

Server certificate → signed by intermediate CA → signed by root CA. The client trusts the root CA (pre-installed in OS/browser trust store). The chain proves the server is who it claims to be. If any link is broken or expired, the connection fails.

mTLS (Mutual TLS)

Standard TLS: only the server presents a certificate. mTLS: both client AND server present certificates. Both sides verify each other's identity. Used in service-to-service communication (service mesh), API authentication, and zero-trust architectures. See Q52 for deep coverage.

Q5 Encryption - symmetric, asymmetric, hashing, PKI

Symmetric Encryption

One key for both encryption and decryption. Fast. Used for bulk data encryption. Algorithms: AES-256 (the standard), ChaCha20. Challenge: how do you securely share the key? TLS solves this - the handshake uses asymmetric crypto to exchange the symmetric key.

Asymmetric Encryption

Two keys: public key encrypts, private key decrypts. Slower but solves the key distribution problem. Algorithms: RSA, ECDSA, Ed25519. Used for: TLS handshakes, digital signatures, SSH authentication.

Hashing

One-way function: input → fixed-length output. Cannot be reversed. Used for: password storage (bcrypt, argon2 - never MD5 or SHA1), file integrity verification (SHA-256 checksums), digital signatures.

PKI (Public Key Infrastructure)

The trust framework for certificates. Certificate Authorities (CAs) verify identity and issue certificates. Root CAs are pre-trusted by operating systems and browsers. Intermediate CAs issue server certificates. The chain of trust from server cert → intermediate → root CA enables TLS verification.

The encryption at rest vs in transit distinction: At rest = data stored on disk (EBS encryption, S3 SSE, database encryption). In transit = data moving over networks (TLS/HTTPS). Production systems must encrypt BOTH.

Q6 OWASP Top 10 - SQLi, XSS, CSRF, SSRF

The Simple Answer

The OWASP Top 10 lists the most critical web application security risks. Every developer and DevOps engineer should know them.

A01: Broken Access Control - Users accessing resources they should not. Missing authorization checks, IDOR (Insecure Direct Object References).

A02: Cryptographic Failures - Weak encryption, plaintext credentials, expired certificates, MD5/SHA1 for passwords.

A03: Injection - SQL injection, command injection, LDAP injection. User input executed as code. Fix: parameterized queries, input validation.

A04: Insecure Design - Security not considered during design. No threat modeling. Missing security requirements.

A05: Security Misconfiguration - Default credentials, unnecessary features enabled, verbose error messages, missing security headers.

A06: Vulnerable Components - Using libraries with known CVEs. Fix: dependency scanning, regular updates.

A07: Authentication Failures - Weak passwords, missing MFA, credential stuffing, session fixation.

A08: Software and Data Integrity - Unverified updates, unsigned packages, CI/CD pipeline manipulation.

A09: Logging Failures - No audit trail, no alerting on suspicious activity, logs not protected.

A10: SSRF - Server-Side Request Forgery. Application makes requests to internal resources based on user input. Attacker accesses metadata services, internal APIs, or databases through the vulnerable application.

Q7 Secret management - Vault, Secrets Manager

The Simple Answer

Secrets (passwords, API keys, certificates, tokens) must never be in source code, environment variables, config files, or Docker images. They belong in a dedicated secret management system.

HashiCorp Vault

The industry standard. Features: dynamic secrets (generate unique credentials per consumer), automatic rotation, fine-grained access policies, audit logging of all secret access, encryption as a service. Architecture: Vault server stores encrypted secrets. Clients authenticate (AppRole, Kubernetes auth, OIDC) and read secrets they are authorized to access.

AWS Secrets Manager

Managed secret storage with automatic rotation for RDS, Redshift, and DocumentDB passwords. Cross-account and cross-region replication. Lambda-based rotation for custom secrets.

Sealed Secrets (Kubernetes)

Encrypt secrets in Git using a cluster-specific public key. Only the cluster's sealed secrets controller can decrypt them. Enables GitOps for secrets - encrypted secrets committed to Git, decrypted only in the cluster.

The hierarchy of secret management maturity: Level 0: Secrets in code (terrible). Level 1: Environment variables (better). Level 2: CI platform secrets (good). Level 3: Vault/Secrets Manager with rotation (great). Level 4: Dynamic short-lived credentials with OIDC (best).

Q8 SSH security - key types, rotation, bastion hosts

The Simple Answer

Key types: ed25519 (recommended - faster, smaller, more secure than RSA), RSA-4096 (legacy but widely supported). Never use DSA or RSA-1024.

Key rotation: Rotate SSH keys every 90 days. Remove keys from servers when employees leave. Use a centralized key management system.

Bastion hosts (jump boxes): A hardened server in a public subnet that provides SSH access to private subnet servers. Users SSH to the bastion, then SSH to the target. The target server is never directly accessible from the internet. Modern alternative: AWS SSM Session Manager - no bastion needed, no port 22, full audit trail.

SSH certificates: Instead of distributing public keys to every server, use an SSH CA. The CA signs short-lived certificates for users. Servers trust the CA, not individual keys. Benefits: automatic expiration, centralized access control, no authorized_keys management.

Q9 Firewall and network security - WAF, IDS/IPS, DMZ

The Simple Answer

Firewall - Controls traffic flow based on rules (allow/deny by source, destination, port, protocol). Network firewalls operate at L3/L4. Web Application Firewalls (WAF) operate at L7 and inspect HTTP content.

IDS (Intrusion Detection System) - Monitors traffic for suspicious patterns and alerts. Passive - does not block.

IPS (Intrusion Prevention System) - Monitors AND blocks suspicious traffic. Active - can drop malicious packets.

DMZ (Demilitarized Zone) - A network segment between the public internet and the private network. Public-facing servers (web servers, load balancers) sit in the DMZ. Internal servers (databases, application servers) sit in the private network. Similar to the public/private subnet pattern in cloud VPCs.

Zero Trust Networking

Traditional: trust everything inside the network perimeter. Zero trust: trust nothing, verify everything. Every request must be authenticated and authorized regardless of network location. See Q51 for deep coverage.

Q10 Vulnerability scanning - CVEs, CVSS scores, patching

The Simple Answer

CVE (Common Vulnerabilities and Exposures) - A unique identifier for a known vulnerability. CVE-2021-44228 is Log4Shell. CVEs are published in the National Vulnerability Database (NVD).

CVSS (Common Vulnerability Scoring System) - Scores vulnerability severity from 0-10. Critical: 9.0-10.0. High: 7.0-8.9. Medium: 4.0-6.9. Low: 0.1-3.9. The score considers: how easy is it to exploit, what access does it require, what impact does it have.

Patch management: Scan regularly (daily for critical systems). Prioritize by CVSS score AND exploitability (see Q48 for EPSS). Patch critical vulnerabilities within 24-48 hours. Have a patching process that includes testing before production deployment.

Q11 Dependency scanning - SCA, Dependabot, Renovate

The Simple Answer

Software Composition Analysis (SCA) scans your application's dependencies (npm packages, Python libraries, Java JARs) against known vulnerability databases. Most applications are 80%+ third-party code - if a dependency has a vulnerability, your application is vulnerable.

Dependabot - GitHub native. Automatically creates PRs to update vulnerable dependencies. **Renovate** - More configurable. Supports grouping updates, custom schedules, auto-merge for minor updates. **Snyk** - Commercial SCA with fix suggestions and priority scoring.

Best Practices

Run SCA on every PR. Block merges on critical/high vulnerabilities. Use lock files (package-lock.json, Pipfile.lock) to pin versions. Auto-merge patch updates for trusted dependencies. Review major updates manually.

Q12 Container image scanning - Trivy, Grype, Docker Scout

The Simple Answer

Container image scanners check every package and library in a Docker image against CVE databases. They find vulnerabilities in: OS packages (apt/yum), application libraries (npm/pip/maven), and the base image itself.

Trivy - Most popular open-source scanner. Fast. Scans images, filesystems, Git repos, Kubernetes clusters, and IaC. **Grype** - Anchore's open-source scanner. Fast, focuses on image scanning. **Docker Scout** - Built into Docker Desktop and Docker Hub. Continuous monitoring with remediation advice.

Integration

Scan in CI on every build. Block pushes of images with critical vulnerabilities. Scan running images in the registry continuously - new CVEs are published daily. A clean scan today does not mean clean tomorrow.

Q13 Git security - signed commits, branch protection, git-secrets

The Simple Answer

Signed commits - Cryptographically prove who made a commit. Uses GPG or SSH keys. GitHub shows a "Verified" badge. Required in some compliance environments to prove code provenance.

Branch protection - Enforce rules on critical branches. Require PR reviews, passing CI, signed commits, and no force pushes. Never allow direct pushes to main.

git-secrets - Pre-commit hook that scans for patterns matching AWS keys, passwords, and other secrets. Prevents accidental commits of credentials. Also: gitleaks, trufflehog for post-commit scanning.

Pre-commit hooks - Run checks before every commit. Lint code, scan for secrets, validate YAML. Install with pre-commit framework. Catches issues at the earliest possible point - the developer's machine.

Q14 Least privilege - IAM, RBAC, service accounts

The Simple Answer

Grant only the minimum permissions needed to perform a task. Start with zero permissions and add as needed. Never start with admin and restrict later - you will forget to restrict something.

How to Implement

Users - Group-based permissions. Never attach policies directly to users. **Services** - Dedicated service accounts with specific roles. Never use the default service account. **CI/CD** - OIDC instead of stored credentials. Short-lived tokens. **Infrastructure** - IAM roles for EC2/ECS/Lambda. No access keys baked into instances.

The test: "If this identity were compromised, what is the blast radius?" If the answer is "everything," the identity has too many permissions. If the answer is "only the payment service's read access to its own database," that is least privilege.

Q15 Logging and audit trails

The Simple Answer

Audit logging records WHO did WHAT, WHEN, and WHERE. It is essential for security investigation, compliance, and incident response. Without audit logs, you cannot investigate breaches, prove compliance, or understand what happened during an incident.

What to Log

Authentication events (login success/failure, MFA usage). Authorization decisions (access granted/denied). Data access (who read/wrote sensitive data). Configuration changes (who modified security groups, IAM policies). Administrative actions (user creation, permission changes).

Protection

Audit logs must be tamper-proof. Store in a separate account. Use write-once storage (S3 Object Lock). Enable log file integrity validation (CloudTrail). Alert on log deletion attempts.

Q16 Multi-factor authentication

The Simple Answer

MFA requires two or more verification factors: something you know (password), something you have (phone, hardware key), something you are (fingerprint, face). Even if a password is compromised, the attacker cannot access the account without the second factor.

Methods (Security Ranking)

Hardware security keys (FIDO2/WebAuthn) - Most secure. Phishing-resistant. Cannot be remotely intercepted. YubiKey, Google Titan. **Authenticator apps (TOTP)** - Google Authenticator, Authy. Time-based codes. Secure against network interception but vulnerable to phishing. **Push notifications** - Duo, Microsoft Authenticator. User approves on their device. Vulnerable to push fatigue attacks (spamming approval requests). **SMS** - Least secure. Vulnerable to SIM swapping. Better than nothing but should not be the only MFA option.

Production rule: MFA on every human account. Hardware keys for admin access. TOTP minimum for developers. SMS only as a fallback.

Q17 Password policies and credential hygiene

The Simple Answer

Password hashing - Never store plaintext passwords. Use bcrypt or argon2 (includes salt, configurable work factor). Never MD5 or SHA-256 for passwords - they are too fast, enabling brute force.

Password policies - Minimum 12 characters. Do not enforce complexity rules (uppercase, special chars) - they lead to predictable patterns (Password1!). Encourage passphrases (correct-horse-battery-staple). Check against breached password databases (Have I Been Pwned).

Credential rotation - Rotate service credentials every 90 days. API keys every 180 days. Use dynamic credentials (Vault) that expire automatically. Never share credentials between environments.

Q18 Network segmentation and DNS security

Network Segmentation

Divide the network into zones with controlled access between them. Public zone (internet-facing), application zone (internal services), data zone (databases), management zone (admin tools). Traffic between zones passes through firewalls. A breach in the public zone cannot reach the database zone directly.

DNS Security

DNSSEC - Signs DNS responses to prevent DNS spoofing. Verifies that DNS responses came from the authoritative server and were not modified in transit.

DNS over HTTPS/TLS (DoH/DoT) - Encrypts DNS queries to prevent eavesdropping. Without encryption, anyone on the network can see which domains you query.

DNS exfiltration - Attackers encode stolen data in DNS queries (data.attacker.com). DNS traffic often bypasses firewalls because it is "trusted." Monitor DNS query patterns for anomalies - unusual query volumes, long subdomain names, queries to known-bad domains.

Q19 Compliance - SOC 2, ISO 27001, PCI DSS, HIPAA, GDPR

The Simple Answer

SOC 2 - Trust services criteria: security, availability, processing integrity, confidentiality, privacy. Annual audit. Required by most enterprise customers before buying SaaS.

ISO 27001 - International information security management standard. Requires an ISMS (Information Security Management System). Certification valid for 3 years with annual surveillance audits.

PCI DSS - Payment Card Industry Data Security Standard. Required for any organization handling credit card data. Prescriptive controls for network security, encryption, access control, and monitoring.

HIPAA - Health Insurance Portability and Accountability Act. Protects patient health information (PHI). Required for healthcare organizations and their technology providers.

GDPR - General Data Protection Regulation. EU data privacy law. Right to be forgotten, data portability, breach notification within 72 hours, consent requirements. Applies to any organization processing EU resident data.

Q20 Security in CI/CD - where to add security gates

The Simple Answer

Security gates should be integrated throughout the pipeline, not bolted on at the end.

The Security Pipeline

Pre-commit - git-secrets, gitleaks (catch secrets before they enter the repo). IDE security linting.

PR/Build - SAST (Semgrep, CodeQL - find code vulnerabilities). SCA (Snyk, Dependabot - find vulnerable dependencies). IaC scanning (tfsec, checkov - find infrastructure misconfigurations). License scanning (FOSSA - catch GPL violations).

Post-build - Container image scanning (Trivy - find OS/library vulnerabilities). Image signing (cosign - prove provenance).

Pre-deploy - Admission webhooks (block unsigned images, enforce security contexts). Policy checks (OPA/Kyverno - verify Kubernetes manifests).

Post-deploy - DAST (OWASP ZAP - test running application). Runtime security (Falco - detect anomalous behavior).

The principle: Fast checks (lint, SAST) run on every commit. Slower checks (DAST, pen test) run nightly or on staging. Never skip any check - make them fast, not optional.

SECTION 2 - Mid-Level Implementation (Q21-Q50)

Q21 SAST - static application security testing

The Simple Answer

SAST analyzes source code (or bytecode) for security vulnerabilities without executing the application. Runs early in the pipeline - during build or even in the IDE. Catches: SQL injection, XSS, command injection, hardcoded secrets, insecure deserialization, buffer overflows.

Tools

Semgrep - Open-source, rule-based. Custom rules in YAML. Fast. Growing rapidly. Best for DevOps teams that want to write custom security rules. **CodeQL** - GitHub's semantic analysis engine. Powerful but complex. Best for deep analysis. Free for public repos. **SonarQube** - Code quality + security. Broad language support. Dashboard with trends. Commercial with free community edition.

False Positives

SAST tools produce false positives - flagging code as vulnerable when it is not. Rate varies 10-50% depending on tool and codebase. Triage process: mark false positives, add suppressions with justification comments, review suppressions quarterly. Never disable SAST because of false positives - tune it.

The practical integration: Run SAST on every PR. Block merges on critical/high findings. Review medium findings weekly. Track false positive rate and tune rules to reduce it.

Q22 DAST - dynamic testing, OWASP ZAP

The Simple Answer

DAST tests the running application by sending malicious requests - SQL injection payloads, XSS scripts, authentication bypass attempts. It finds vulnerabilities that SAST misses because it tests actual behavior, not just code patterns.

OWASP ZAP - Free, open-source. Automated scanning and manual testing. API scanning. CI/CD integration via CLI. The standard for open-source DAST. **Burp Suite** - Commercial. Industry standard for penetration testers. More powerful manual testing than ZAP.

Authenticated Scanning

DAST needs to log in to test authenticated endpoints. Configure: login URL, credentials (stored securely), session handling. Without authentication, DAST only tests public-facing pages - missing the majority of the application.

Integration

Run DAST against staging (not production - it sends malicious payloads). Schedule nightly rather than per-commit (too slow). Alert on critical findings. Track trends over time.

Q23 SCA and SBOM - software composition analysis

The Simple Answer

SCA scans your dependency tree against known vulnerability databases. An SBOM (Software Bill of Materials) is a complete inventory of every component in your software - like a nutrition label for software.

SBOM Formats

SPDX - Linux Foundation standard. **CycloneDX** - OWASP standard. Both list: component name, version, license, supplier, and known vulnerabilities. Generate with: Syft (images), cdxgen (source code), or language-specific tools.

Why SBOMs Matter

Executive Order 14028 (US) requires SBOMs for software sold to the federal government. Customers increasingly request SBOMs as part of vendor security assessment. When a new CVE drops (like Log4Shell), an SBOM instantly answers "are we affected?" without manual investigation.

Q24 IaC security - tfsec, checkov, kics

The Simple Answer

IaC security tools scan Terraform, CloudFormation, Kubernetes manifests, and Docker files for misconfigurations before deployment. They catch: public S3 buckets, unencrypted databases, overly permissive security groups, missing logging, and hundreds of other issues.

tfsec - Terraform-focused. Fast. Now part of Trivy. **checkov** - Multi-framework (Terraform, CloudFormation, K8s, Docker, ARM, Bicep). Most comprehensive. Custom policies in Python. **KICS** - Checkmarx's open-source IaC scanner. Broad framework support.

Integration

Run in CI on every PR that modifies infrastructure code. Block merges on critical findings (public database, no encryption). Treat IaC security scans the same as SAST - they catch vulnerabilities before deployment.

Q25 JWT security - claims, signing, attacks

The Simple Answer

JWT (JSON Web Token) is a compact, self-contained token for authentication and information exchange. Three parts: Header (algorithm, type) . Payload (claims - sub, exp, iat, custom) . Signature (verifies integrity).

Signing Algorithms

HS256 - Symmetric. Same secret signs and verifies. Simple but the secret must be shared between issuer and verifier. **RS256** - Asymmetric. Private key signs, public key verifies. The verifier never needs the private key. Preferred for distributed systems. **None** - No signature. NEVER accept alg: none in production - it is the #1 JWT attack vector.

Common Attacks

Algorithm confusion - Attacker changes alg from RS256 to HS256. The server uses the public key (intended for RS256 verification) as the HS256 secret. Since the public key is public, the attacker can forge tokens. Fix: always validate the algorithm server-side, never trust the token's alg header.

Token expiration bypass - No exp claim or very long expiration. Fix: always set exp to a short duration (15-60 minutes). Use refresh tokens for longer sessions.

Token theft - XSS steals the JWT from localStorage. Fix: store in httpOnly cookies (not accessible to JavaScript). Use short expiration. Implement token binding.

Q26 Container security hardening

The Hardening Checklist

Run as non-root - USER instruction in Dockerfile. runAsNonRoot: true in Kubernetes securityContext.

Read-only root filesystem - readOnlyRootFilesystem: true. Application writes only to explicitly mounted volumes. Prevents attackers from modifying binaries.

Drop all capabilities - capabilities: drop: ["ALL"]. Add back only what is needed (rarely anything for web applications).

No privilege escalation - allowPrivilegeEscalation: false.

Minimal base image - Distroless or scratch. Fewer packages = fewer vulnerabilities = smaller attack surface.

Scan images - Trivy in CI. Block images with critical CVEs.

No secrets in images - Use BuildKit secret mounts. Never COPY .env files.

The hardened securityContext that every production Pod should have: runAsNonRoot: true, readOnlyRootFilesystem: true, allowPrivilegeEscalation: false, capabilities: drop: ["ALL"]. This single block prevents entire classes of container attacks.

Q27 Kubernetes security - RBAC, PSA, NetworkPolicies

RBAC

Least privilege for all ServiceAccounts. Dedicated SA per application. Never use cluster-admin for automation. Audit permissions regularly with kubectl auth can-i.

Pod Security Admission (PSA)

Enforce Restricted profile on production namespaces. Audit and warn before enforcing. The Restricted profile requires: non-root, read-only root filesystem, no privileged containers, no hostNetwork, drop all capabilities.

NetworkPolicies

Default-deny in every namespace. Explicitly allow only required communication. Include egress rules - a compromised Pod should not be able to call arbitrary external IPs. Allow DNS (port 53) in egress policies.

Secrets

Base64 is not encryption. Enable etcd encryption at rest. Use External Secrets Operator with Vault/AWS Secrets Manager. Never store secrets in ConfigMaps.

Q28 Image signing - cosign, Notary, admission webhooks

The Simple Answer

Image signing cryptographically proves that an image was built by a trusted pipeline and has not been tampered with. Admission webhooks verify signatures before allowing images to run in the cluster.

cosign (Sigstore)

Signs container images using keyless signing with OIDC identity. CI pipeline signs the image during build. The cluster verifies the signature before deployment. No long-lived signing keys to manage. The signature is stored alongside the image in the registry.

Admission Webhook Verification

Kyverno or Connaisseur webhook checks that every image has a valid signature before allowing the Pod to be created. Unsigned images are rejected. This ensures only images built by your CI pipeline run in production - not images pushed manually or from compromised registries.

Q29 Supply chain security - SLSA, Sigstore

SLSA (Supply-chain Levels for Software Artifacts)

Level 1 - Documentation of the build process. **Level 2** - Build on a hosted service (not developer laptops). **Level 3** - Hardened build platform. Tamper-proof provenance attestation. Non-falsifiable - the build system signs the provenance, not the developer. **Level 4** - Two-party review. Hermetic builds (no network access during build).

Sigstore

cosign - Signs artifacts. **Rekor** - Transparency log (immutable record of all signatures). **Fulcio** - Issues short-lived signing certificates tied to OIDC identity. Together: CI builds an image → Fulcio issues a certificate based on the CI platform's OIDC identity → cosign signs the image → Rekor records the signature → admission webhook verifies the chain before deployment.

Q30 Secret rotation and dynamic secrets

Dynamic Secrets

Vault generates unique, short-lived credentials for each consumer. A CI pipeline requests database credentials → Vault creates a new database user with specific permissions → credentials expire after the pipeline completes → the user is automatically deleted. No shared credentials. No rotation needed. Every consumer gets unique credentials.

Rotation Strategies

Automatic rotation - AWS Secrets Manager rotates RDS passwords automatically via Lambda. Vault rotates credentials on a schedule. **Blue-green rotation** - Create a new credential. Update consumers. Verify the old credential is unused. Delete the old credential. Zero downtime if both credentials are valid during the transition. **Forced rotation after incident** - Rotate everything immediately. Accept brief disruption. Faster rotation is more important than graceful rotation during a compromise.

Q31 CORS deep dive - preflight, origins, misconfigurations

The Simple Answer

CORS (Cross-Origin Resource Sharing) controls which websites can make requests to your API from a browser. Without CORS, a malicious website could make authenticated requests to your API using the user's cookies.

How It Works

Browser sends a preflight OPTIONS request asking "can origin X access this resource?" Server responds with Access-Control-Allow-Origin, allowed methods, and allowed headers. If the origin is allowed, the browser sends the actual request.

Dangerous Misconfigurations

Access-Control-Allow-Origin: * - Allows ANY website to access your API. Dangerous if combined with cookies/credentials. **Reflecting the Origin header** - Dynamically setting ACAO to whatever origin the request came from. Equivalent to * but bypasses the browser's restriction on * with credentials. **Allow-Credentials: true with wide origin** - Lets any website make authenticated requests as the logged-in user.

The fix: Whitelist specific, trusted origins. Never use * for APIs with authentication. Validate the Origin header against a strict allowlist.

Q32 API security - rate limiting, validation, OAuth vs keys

Rate Limiting

Prevent abuse: 100 requests per minute per API key. 10 login attempts per minute per IP. Return 429 Too Many Requests when exceeded. Implement at the API gateway level (not in application code). Use token bucket or sliding window algorithms.

Input Validation

Validate every input: type, length, format, range. Reject unexpected fields. Use schema validation (JSON Schema, OpenAPI). Never trust client input - validate server-side even if you validate client-side.

OAuth vs API Keys

API keys - Simple. Identify the caller. No user context. Easy to leak. Good for: server-to-server calls, public APIs with rate limiting. **OAuth** - Complex. Authenticate the user and authorize specific scopes. Short-lived tokens. Better security. Good for: user-facing APIs, fine-grained authorization.

Q33 CSP and security headers

The Simple Answer

Security headers instruct the browser to enable security features that protect against common attacks.

Content-Security-Policy (CSP) - Controls which resources the browser can load. Prevents XSS by blocking inline scripts and unauthorized external scripts. **Strict-Transport-Security (HSTS)** - Forces HTTPS. Prevents downgrade attacks. **X-Content-Type-Options: nosniff** - Prevents MIME type sniffing. **X-Frame-Options: DENY** - Prevents clickjacking (your page loaded in an iframe). **Referrer-Policy** - Controls what URL information is sent in the Referrer header.

Q34 Key management - HSMs, rotation, envelope encryption

HSM (Hardware Security Module)

Dedicated hardware for cryptographic operations. Keys never leave the HSM - they are generated, stored, and used inside the tamper-resistant device. AWS CloudHSM, GCP Cloud HSM, Azure Dedicated HSM. Used for: PCI DSS compliance, certificate authority keys, high-value encryption keys.

Envelope Encryption

Encrypt data with a data key. Encrypt the data key with a master key (KMS). Store the encrypted data key alongside the data. To decrypt: KMS decrypts the data key, the data key decrypts the data. Benefits: the master key never leaves KMS, large data volumes are encrypted locally (fast), and key rotation only requires re-encrypting the data key (not all the data).

Q35 Cloud security posture management

The Simple Answer

CSPM tools continuously scan cloud environments for misconfigurations: public S3 buckets, unencrypted databases, overly permissive security groups, disabled logging, missing MFA. Tools: AWS Security Hub + Config, GCP Security Command Center, Azure Defender for Cloud, Prisma Cloud (Palo Alto), Wiz.

Why CSPM Matters

Cloud misconfigurations are the #1 cause of data breaches in cloud environments. Not sophisticated attacks - just open S3 buckets, public databases, and overly permissive IAM policies. CSPM catches these continuously, not just at deployment time.

Q36 Penetration testing

Types

Black box - Tester has no knowledge of the system. Simulates an external attacker. **Gray box** - Tester has partial knowledge (documentation, limited credentials). Most realistic for insider threat simulation. **White box** - Tester has full knowledge (source code, architecture, credentials). Most thorough. Finds the most vulnerabilities.

Cloud Provider Rules

AWS, GCP, and Azure allow penetration testing without prior approval for most services. Some services are excluded (DDoS testing, DNS zone walking). Check each provider's pen testing policy before starting.

Q37 Threat modeling - STRIDE, DREAD

STRIDE

Categorize threats: **S**poofing (impersonating someone), **T**ampering (modifying data), **R**epudiation (denying actions), **I**nformation disclosure (data leaks), **D**enial of service, **E**levation of privilege. For each component in your architecture, ask "can an attacker achieve S, T, R, I, D, or E?"

The Process

1. Draw a data flow diagram of your system. 2. Identify trust boundaries (where do security contexts change?). 3. For each component and data flow, apply STRIDE. 4. Prioritize threats by likelihood and impact. 5. Define mitigations for each threat. 6. Review and update quarterly or after major changes.

Q38 Secrets sprawl detection

The Simple Answer

Secrets sprawl occurs when credentials appear in places they should not: Git history, CI logs, Slack messages, Confluence pages, Docker images, config files on servers, developer laptops.

Detection Tools

gitleaks - Scans Git repositories (including history) for secrets. **trufflehog** - Deep Git history scanning with high accuracy. **detect-secrets (Yelp)** - Baseline-based approach. Detects new secrets without flagging existing ones. Run in CI pre-commit.

Remediation

When a secret is found: rotate immediately (assume compromised), remove from Git history (git filter-repo), scan all other locations (CI logs, Slack, wikis), and add pre-commit hooks to prevent recurrence.

Q39 Security monitoring - SIEM, SOC

SIEM (Security Information and Event Management)

Aggregates logs from all sources (servers, applications, network devices, cloud services), correlates events, detects patterns, and alerts on threats. Examples: Splunk, Elastic SIEM, Chronicle (Google), Sentinel (Azure), CloudWatch + GuardDuty (AWS).

SOC (Security Operations Center)

A team (or managed service) that monitors SIEM alerts 24/7, investigates incidents, and coordinates response. SOC analysts use SIEM dashboards, threat intelligence feeds, and playbooks to detect and respond to threats.

Detection Rules

Write rules for: brute force attempts (10+ failed logins in 1 minute), privilege escalation (user granted admin), impossible travel (login from two countries within 1 hour), data exfiltration (large download from database), and configuration changes (security group modified, encryption disabled).

Q40 Incident response plan

The Six Phases

1. Preparation - IR plan documented. Team trained. Tools ready. Runbooks written. Contact lists current.

2. Identification - Detect the incident. Alert fires, user reports, or threat intelligence tip.

3. Containment - Stop the spread. Short-term: isolate affected systems. Long-term: apply patches, change credentials.

4. Eradication - Remove the threat. Delete malware, close vulnerabilities, remove attacker access.

5. Recovery - Restore normal operations. Rebuild compromised systems from clean images. Restore data from backups. Monitor closely for re-compromise.

6. Lessons Learned - Blameless postmortem. What happened? How did we detect it? How do we prevent it? Update the IR plan.

Q41 Certificate management at scale

cert-manager in Kubernetes

Automates TLS certificate lifecycle. Define an Issuer (Let's Encrypt, Vault, internal CA) and a Certificate resource. cert-manager obtains, stores (as Kubernetes Secret), and renews certificates automatically. Integrates with Ingress controllers for automatic HTTPS.

Internal CAs

For service-to-service mTLS, use an internal CA (Vault PKI, AWS ACM Private CA). Issue short-lived certificates (24 hours). Automatic rotation eliminates certificate expiry incidents. SPIFFE/SPIRE provides identity-based certificate issuance for workloads.

Q42 K8s network policies deep dive

Default Deny Pattern

Apply a policy selecting all Pods with empty ingress and egress rules. All traffic is blocked. Then add specific allow policies for each communication path. This is zero-trust for Kubernetes networking.

Egress Control

Control outbound traffic from Pods. Allow DNS (port 53 to kube-dns). Allow specific external IPs or CIDRs. Block everything else. A compromised Pod cannot call attacker-controlled servers if egress is restricted.

DNS Policies

Allow DNS resolution (port 53 UDP/TCP to kube-system namespace). Without this, Pods cannot resolve any service names. This is the most commonly forgotten rule when implementing default-deny.

Q43 Secrets in Kubernetes - ESO, sealed secrets, Vault agent

External Secrets Operator (ESO)

Syncs secrets from external stores (Vault, AWS Secrets Manager, GCP Secret Manager) into Kubernetes Secrets. The external store is the source of truth. ESO polls for changes and updates the K8s Secret automatically.

Sealed Secrets

Encrypt secrets client-side with a cluster-specific public key. Commit the encrypted SealedSecret to Git. The sealed-secrets controller in the cluster decrypts it. Enables GitOps for secrets.

Vault Agent Injector

Mutating webhook that injects a Vault sidecar into Pods. The sidecar authenticates to Vault (Kubernetes auth), retrieves secrets, and writes them to a shared volume. The application reads secrets from the filesystem. No K8s Secret object needed - secrets go directly from Vault to the Pod.

Q44 Docker security - daemon, socket, Bench

Daemon Security

Docker daemon runs as root. Never expose the Docker API over TCP without TLS. Never mount /var/run/docker.sock into untrusted containers (gives full host control). Use rootless mode when possible.

Docker Bench for Security

Automated CIS benchmark checks for Docker. Covers: host configuration, daemon configuration, container images, container runtime, Docker security operations. Run regularly and track findings over time.

Q45 IAM hardening - cross-account, boundaries, SCPs

The Hardening Checklist

Eliminate root account usage (MFA, strong password, never used for daily work). Enforce MFA for all human users. Remove unused users, roles, and access keys. Use permission boundaries for delegated user creation. SCPs to prevent disabling security controls (CloudTrail, encryption). Use IAM Access Analyzer to find unused permissions. Generate least-privilege policies from CloudTrail usage data.

Q46 Cloud security tools

The Simple Answer

AWS: GuardDuty (threat detection), Security Hub (findings aggregation), Inspector (vulnerability scanning), Macie (data classification), Config (compliance). **GCP:** Security Command Center (unified security), Binary Authorization (image verification), VPC Service Controls (data perimeter). **Azure:** Defender for Cloud (CSPM + CWPP), Sentinel (SIEM), Key Vault (secrets/keys/certs).

Q47 WAF tuning

The Simple Answer

WAF rules block malicious traffic but can also block legitimate traffic (false positives). Tuning is balancing security with availability.

Tuning Process

Deploy in count/log mode first (log matches but do not block). Analyze logs for false positives. Exclude specific rules or paths that generate false positives. Switch to block mode after tuning. Monitor continuously - new application features may trigger existing rules.

Q48 Vulnerability prioritization - EPSS, reachability

Beyond CVSS

CVSS measures severity but not exploitability. A CVSS 9.8 vulnerability in a library function your application never calls is lower priority than a CVSS 7.0 in a function called on every request.

EPSS (Exploit Prediction Scoring System)

Predicts the probability a vulnerability will be exploited in the wild within 30 days. Score 0-1. EPSS 0.97 means 97% chance of exploitation - patch immediately. EPSS 0.01 means 1% chance - schedule for next patch cycle.

Reachability Analysis

Analyzes whether your code actually calls the vulnerable function in the dependency. If the vulnerable code path is unreachable from your application, the risk is lower. Tools: Snyk, Endor Labs provide reachability analysis.

The prioritization formula: Fix first: CVSS Critical + EPSS > 0.5 + reachable. Fix next: CVSS High + EPSS > 0.3. Schedule: everything else. This approach reduces patch urgency by 60-80% without increasing risk.

Q49 Security benchmarks - CIS

The Simple Answer

CIS (Center for Internet Security) Benchmarks provide hardening guidelines for every platform: Linux (Ubuntu, RHEL, Amazon Linux), Docker, Kubernetes, AWS, GCP, Azure. Each benchmark contains hundreds of specific recommendations with remediation steps.

Automated Scanning

kube-bench - CIS Kubernetes benchmark checks. **Docker Bench for Security** - CIS Docker benchmark. **Lynis** - Linux system hardening auditor. **Prowler** - AWS CIS benchmark checks. **ScoutSuite** - Multi-cloud security auditing.

Q50 API gateway security

The Simple Answer

The API gateway is the front door - it is the best place to enforce security controls consistently across all APIs.

Security Controls at the Gateway

Authentication - Validate JWT tokens before requests reach backend services. Reject invalid/expired tokens at the edge. **Rate limiting** - Per-client, per-endpoint throttling. Prevents abuse and DDoS. **Input validation** - Validate request schema against OpenAPI spec. Reject malformed requests. **mTLS** - Verify client certificates for service-to-service calls. **WAF integration** - Inspect request bodies for injection attacks. **IP whitelisting** - Restrict access to known CIDR ranges for internal APIs.

SECTION 3 - Senior Architecture & Advanced (Q51-Q70)

Q51 Zero trust architecture

The Simple Answer

Zero trust means no implicit trust based on network location. Every request is authenticated, authorized, and encrypted - whether it comes from inside or outside the network. "Never trust, always verify."

Implementation Pillars

Identity - Every user and service has a verified identity. MFA for humans. mTLS certificates for services. Short-lived tokens, not long-lived credentials.

Device - The device accessing the resource must be healthy and compliant. MDM verification, OS patching status, endpoint detection.

Network - Microsegmentation. NetworkPolicies. No flat networks. Every connection is encrypted (mTLS). No VPN-based trust - VPN provides connectivity, not authorization.

Application - Every API endpoint checks authorization. No "internal-only" APIs without auth. Service mesh enforces mTLS between all services.

Data - Classification. Encryption at rest and in transit. DLP rules. Access logging.

Google BeyondCorp

Google's implementation: access decisions based on user identity + device state + request context, not network location. Employees access internal applications from any network (no VPN needed). Each request is evaluated by an access proxy that checks identity, device health, and contextual signals.

Q52 mTLS everywhere - SPIFFE/SPIRE

The Simple Answer

mTLS encrypts all service-to-service traffic AND verifies both sides' identities. In a service mesh (Istio/Linkerd), mTLS is automatic - the sidecar proxy handles certificate management, rotation, and verification. No application code changes.

SPIFFE/SPIRE

SPIFFE - Secure Production Identity Framework for Everyone. Defines a standard identity format (SPIFFE ID) for workloads: `spiffe://trust-domain/path`. **SPIRE** - The runtime that issues and manages SPIFFE identities. Issues short-lived X.509 certificates (SVIDs) to workloads. Automatic rotation (typically every hour). Works across Kubernetes, VMs, and cloud providers.

Certificate Rotation

Short-lived certificates (1-24 hours) eliminate the certificate expiry problem. If a certificate is compromised, it expires within hours without manual revocation. The mesh or SPIRE handles rotation transparently - no application downtime.

Q53 Policy as code - OPA, Kyverno, Sentinel

OPA/Gatekeeper

OPA (Open Policy Agent) evaluates policies written in Rego (a purpose-built policy language). Gatekeeper integrates OPA with Kubernetes admission control. Policies: require resource limits, deny privileged containers, enforce image registries, require specific labels. Powerful but Rego has a steep learning curve.

Kyverno

Kubernetes-native policies written in YAML - no new language to learn. Can validate (reject non-compliant resources), mutate (add missing fields automatically), generate (create NetworkPolicies when namespaces are created), and clean up resources. Lower learning curve than OPA. Better for Kubernetes-only policy enforcement.

Sentinel (HashiCorp)

Policy as code for Terraform Enterprise/Cloud. Enforces policies on infrastructure changes before they are applied. Example: deny public S3 buckets, require encryption on all databases, enforce tagging.

Q54 Runtime security - Falco, eBPF

Falco

CNCF runtime security tool. Uses eBPF to monitor system calls in real-time. Detects: unexpected shell access in containers, unauthorized process execution, file reads from sensitive paths (/etc/shadow), network connections to unknown destinations, privilege escalation attempts. Default rules cover common threats. Custom rules for organization-specific detection.

How eBPF Enables Detection

eBPF programs run inside the kernel, observing system calls without overhead. Falco hooks into: exec (process creation), open (file access), connect (network connections), and other syscalls. Every container action is visible regardless of what tools are installed inside the container.

The value: Your container image has no shell, no debugging tools, nothing extra. An attacker exploits a vulnerability and spawns a shell. Traditional monitoring sees nothing (no logs from a non-existent shell). Falco's eBPF hook detects the exec syscall immediately and alerts: "shell spawned in container where shells are not expected."

Q55 Vault HA architecture

Architecture

Vault in HA mode runs multiple servers. One is the active leader, others are standby. All requests go to the leader. Standby servers forward requests to the leader. Storage backend: Consul (traditional), integrated Raft storage (recommended for new deployments - no external dependency).

Auto-Unseal

Vault starts in a sealed state - it cannot read its encrypted data until unsealed with unseal keys. Manual unsealing requires human intervention on every restart. Auto-unseal uses a cloud KMS (AWS KMS, GCP KMS, Azure Key Vault) to unseal automatically. The cloud KMS holds the root key. Vault decrypts its master key using the KMS on startup. No human intervention needed.

Namespaces (Enterprise)

Vault Enterprise supports namespaces - isolated Vault environments within a single cluster. Each team gets their own namespace with independent policies, auth methods, and secret engines. Multi-tenancy without separate Vault clusters.

Q56 RBAC vs ABAC vs ReBAC

RBAC (Role-Based)

Permissions assigned to roles, roles assigned to users. Simple, widely used. Limitation: role explosion in complex organizations - hundreds of roles for slight permission variations.

ABAC (Attribute-Based)

Permissions based on attributes of the user, resource, action, and environment. "Allow if user.department == resource.department AND time < 18:00." Flexible. Handles complex policies without role explosion. AWS IAM conditions are ABAC. Kubernetes does not natively support ABAC well.

ReBAC (Relationship-Based)

Permissions based on relationships between entities. "User can edit document if user is owner of document OR user is member of document's team." Google Zanzibar (used for Google Docs sharing), OpenFGA, SpiceDB. Best for applications where access depends on data relationships (social networks, document sharing, organizational hierarchies).

Q57 Security architecture review

The Process

1. Map the architecture - all services, data flows, trust boundaries, external dependencies. 2. Classify data - what is sensitive? PII, financial, health, credentials. 3. Apply STRIDE at each trust boundary. 4. Evaluate existing controls - encryption, authentication, authorization, logging, network segmentation. 5. Identify gaps - unencrypted data flows, missing auth, overly permissive access, insufficient logging. 6. Prioritize by risk - likelihood × impact. 7. Define mitigations with deadlines and owners. 8. Review quarterly.

Q58 Compliance as code

The Simple Answer

Automate compliance checks so they run continuously, not just during annual audits. AWS Config rules check continuously. Azure Policy denies non-compliant resources in real-time. Custom scripts verify compliance requirements daily. The audit becomes a report generated from always-current data, not a multi-week manual assessment.

Tools

AWS Config + Conformance Packs - Pre-built rule sets for SOC 2, PCI DSS, CIS. **Azure Policy + Blueprints** - Enforce and report on compliance. **Chef InSpec / ScoutSuite** - Infrastructure compliance scanning. **Prowler** - AWS security best practices assessment.

Q59 Data protection - DLP, tokenization, masking

DLP (Data Loss Prevention)

Detects and prevents sensitive data from leaving controlled environments. Scans outbound traffic, emails, uploads, and API responses for patterns matching credit card numbers, SSNs, health records. Blocks or alerts on unauthorized data transfer. Tools: AWS Macie (S3), Google DLP, Microsoft Purview.

Tokenization

Replace sensitive data with non-sensitive tokens. The original data is stored securely in a token vault. The token is useless without access to the vault. Used for: payment card data (PCI DSS), SSNs, health records. Unlike encryption, tokens are not mathematically derived from the original data - even with unlimited compute, you cannot reverse a token.

Data Masking

Replace sensitive data with realistic but fake data for non-production environments. Production database has real PII. Development database has masked PII - same format, different values. Prevents developers from accessing real customer data while maintaining data structure for testing.

Q60 Shift-right security - RASP, canary tokens, honeypots

The Simple Answer

Shift-left catches vulnerabilities before production. Shift-right catches attacks IN production. Both are needed.

RASP (Runtime Application Self-Protection)

An agent inside the application that detects and blocks attacks in real-time. Unlike WAF (inspects HTTP traffic), RASP sees application context - it knows the difference between a SQL query generated by the application and one injected by an attacker. Tools: Contrast Security, Sqreen, Dynatrace RASP.

Canary Tokens

Decoy credentials or files that alert when accessed. Place a fake AWS key in a likely-to-be-attacked location (Git repo, S3 bucket). When an attacker tries to use it, you get an immediate alert. Tools: canarytokens.org (free), Thinkst Canary.

Honeypots

Decoy systems that appear to be real production assets. Attract attackers and provide early warning. Any interaction with a honeypot is malicious by definition - no false positives. Tools: T-Pot, OpenCanary.

Q61 Identity federation - OIDC, SAML, cross-cloud

The Simple Answer

Federation allows a single identity provider to authenticate users across multiple systems and clouds. User authenticates once with their IdP (Okta, Azure AD) and gets access to AWS, GCP, Azure, Kubernetes, and internal applications without separate credentials for each.

Cross-Cloud Identity

AWS: IAM Identity Center with OIDC/SAML to external IdP. GCP: Workload Identity Federation trusts external OIDC tokens. Azure: Entra ID external identities and workload identity federation. CI/CD: GitHub Actions, GitLab CI, Jenkins can authenticate to all three clouds using OIDC without stored credentials.

Q62 Binary authorization - admission controllers

The Simple Answer

Binary authorization ensures only verified, scanned, and signed images run in production. Implementation: CI pipeline builds image → scans for vulnerabilities → signs with cosign → pushes to registry. Kubernetes admission webhook (Kyverno, Connaisseur, or GCP Binary Authorization) checks: is the image signed? By a trusted key? From an approved registry? Did it pass vulnerability scanning? If any check fails, the Pod is rejected.

Policy Chains

Multiple checks in sequence: image from allowed registry → image signed by CI pipeline → image scanned with no critical CVEs → image less than 30 days old. Each policy is an independent admission check. All must pass.

Q63 Privileged access management - PAM, JIT

PAM (Privileged Access Management)

Controls access to high-privilege accounts. Features: credential vaulting (admin passwords stored securely, checked out when needed), session recording (every privileged session is recorded for audit), just-in-time access (permissions granted for a limited time, then automatically revoked). Tools: CyberArk, HashiCorp Boundary, AWS SSM Session Manager.

Just-In-Time (JIT) Access

No standing privileges. Engineers request elevated access for a specific task, for a specific duration. Approval is required. Access is automatically revoked after the time expires. This limits the blast radius of compromised credentials - an attacker who steals a developer's credentials gets read-only access, not admin access.

Q64 Serverless security

Unique Challenges

Function permissions - Each Lambda/Function needs its own IAM role with least privilege. A single overly permissive role shared across functions is a blast radius problem. **Event injection** - Attackers can inject malicious data through event sources (API Gateway, S3, SQS). Validate and sanitize all event data. **Dependency vulnerabilities** - Serverless functions use the same libraries as traditional apps. Scan dependencies in CI. **Cold start + timeout** - Security middleware (WAF, authentication) must not add latency that causes timeouts. **Logging** - Ensure function logs do not contain secrets or PII. Structured logging with correlation IDs.

Q65 Container forensics

Evidence Capture

Do not delete the container. `docker commit container forensics:$(date +%s)` creates an image snapshot. `docker export container > container_fs.tar` captures the filesystem. `docker logs container > logs.txt` captures output. `docker inspect container > metadata.json` captures configuration. Copy all evidence to a secure, tamper-proof location before any remediation.

Analysis

Inspect the filesystem for: modified binaries (compare hashes with original image), new files in `/tmp`, `/dev/shm`, unexpected processes (if captured before stopping), network connections to suspicious IPs, crontab entries and startup script modifications, and reverse shell scripts or cryptocurrency miners.

Q66 Chaos security testing - purple team

Security Game Days

Scheduled exercises where the red team (attackers) attempts to exploit known weaknesses while the blue team (defenders) detects and responds. Verifies: do alerts fire? Do runbooks work? Can the team contain the simulated breach within the target MTTR? Unlike a penetration test (which finds new vulnerabilities), a Game Day tests the response to known scenarios.

Purple Team

Red and blue teams work together rather than adversarially. The red team explains their techniques. The blue team explains their detection gaps. Together they improve both attack simulation and detection capabilities. More efficient than separate red/blue exercises.

Q67 Red team vs blue team

Red Team (Offensive)

Simulates real-world attackers. Uses the same techniques, tools, and tactics (TTPs) as actual threat actors. Goal: find weaknesses before real attackers do. Activities: social engineering, network penetration, application exploitation, physical access testing, supply chain compromise simulation.

Blue Team (Defensive)

Defends the environment. Monitors for threats, responds to incidents, hardens systems, and improves detection. Activities: SIEM monitoring, incident response, vulnerability management, security architecture, compliance.

The Balance

Red team without blue team: finding vulnerabilities nobody fixes. Blue team without red team: building defenses never tested. Both are necessary for a mature security program.

Q68 Container runtime isolation - gVisor, Kata, Firecracker

The Simple Answer

Standard containers (runc) share the host kernel. A kernel vulnerability allows container escape. Stronger isolation options:

gVisor - User-space kernel. Intercepts container system calls and implements them in a sandboxed kernel written in Go. The container never directly calls the host kernel. Performance overhead for syscall-heavy workloads.

Kata Containers - Each container runs in a lightweight VM with its own kernel. Strong isolation (hardware-level). Higher overhead than gVisor. Good for multi-tenant environments running untrusted code.

Firecracker - AWS's lightweight VM monitor (VMM). Powers Lambda and Fargate. Boots in <125ms. Minimal overhead. Strong isolation with near-container performance.

Q69 Bug bounty programs

The Simple Answer

Pay external security researchers to find and report vulnerabilities in your systems. Platforms: HackerOne, Bugcrowd, Synack. Rewards: \$100-\$50,000+ depending on severity and your program. Benefits: diverse testing perspectives, continuous testing (not point-in-time), and motivated researchers who think like attackers.

Running a Program

Define scope clearly (which domains, applications, APIs are in scope). Set clear rules of engagement (no DDoS, no accessing other users' data). Define reward tiers by severity. Respond to reports within 24 hours. Fix critical issues within 7 days. Pay bounties promptly.

Q70 Security culture

Security Champions

Volunteer developers in each team who receive extra security training and act as the first line of security review for their team. They do not replace the security team - they extend it. Each champion spends 10-20% of their time on security activities.

Training

Annual security training for all developers. Quarterly workshops on current threats. Capture-the-flag (CTF) competitions for hands-on practice. Code review checklists that include security items. Gamification: leaderboards for security findings, badges for completing training.

Culture Metrics

Track: time to remediate vulnerabilities (target: critical in 24h, high in 7d), percentage of teams with a security champion, security finding escape rate (how many findings reach production), and security training completion rate.

SECTION 4 - Tricky Security Scenarios (Q71-Q100)

Q71 AWS credentials leaked on GitHub

Step-by-Step

Step 1 - Deactivate the key immediately. IAM → Security Credentials → Deactivate (do not delete yet - needed for audit). This must happen within minutes.

Step 2 - Audit usage with CloudTrail. Search for API calls using the compromised key since the commit timestamp. Look for: EC2 RunInstances (crypto mining), IAM CreateUser/CreateAccessKey (persistence), S3 GetObject (data exfiltration), billing changes, and resource creation in unusual regions.

Step 3 - Revoke active sessions. If the key was used to assume roles, those sessions are still active. Add a deny-all inline policy to the affected entity:

```
{"Effect": "Deny", "Action": "*", "Resource": "*", "Condition": {"DateLessThan": {"aws:TokenIssueTime": "current-time"}}
```

Step 4 - Delete unauthorized resources. Check ALL regions. Attackers launch resources in regions you never monitor (ap-southeast-1, sa-east-1). Check: EC2 instances, Lambda functions, IAM users/roles/keys, EBS volumes, S3 buckets.

Step 5 - Remove from Git history. git filter-repo or BFG Repo Cleaner to remove the commit containing the key. Force push. The key is still in GitHub's cache for a while - contact GitHub support for purging if needed.

Step 6 - Prevent recurrence. Pre-commit hooks (git-secrets, gitleaks). Enable GitHub secret scanning on all repos. Migrate to OIDC - eliminate long-lived access keys entirely. SCP denying access key creation if possible.

Timing matters: AWS monitors GitHub for leaked keys and may proactively notify you or restrict the key. But do not wait for AWS - act within minutes. Automated attackers scan GitHub commits in real-time and can compromise your account within minutes of a leak.

Q72 Container escape - Pod reaching host

Step-by-Step

Step 1 - Isolate immediately. Change the Pod's NetworkPolicy to deny all traffic. If on a dedicated node, cordon and drain the node.

Step 2 - Capture evidence. kubectl logs pod. kubectl exec (if still possible) to check running processes and network connections. Snapshot the node for forensic analysis.

Step 3 - Determine the escape vector. Common vectors: privileged container (full host access), hostPID/hostNetwork (shared namespaces), mounted Docker socket (/var/run/docker.sock), kernel vulnerability exploit, writable hostPath mount.

Step 4 - Assess blast radius. If the attacker reached the host, they potentially accessed: all other containers on the node, kubelet credentials (access to the API server), node filesystem (container images, secrets in tmpfs), and other Pods' network traffic.

Step 5 - Remediate. Terminate the compromised Pod and node. Rotate all credentials accessible from the node. Patch the vulnerability. Review Pod Security Admission - enforce Restricted profile. Block privileged containers via Kyverno/OPA.

Prevention

Pod Security Admission Restricted profile. Never allow privileged containers. Never mount Docker socket. Never use hostPID or hostNetwork. Use gVisor or Kata for untrusted workloads. Runtime

detection with Falco: alert on shell spawned in container, unexpected mount operations, or access to sensitive host paths.

Q73 Critical CVE in production dependency

Step-by-Step

Step 1 - Assess exploitability. Is the vulnerability in a code path your application actually uses (reachability analysis)? What is the EPSS score (probability of exploitation in the wild)? Is there a known exploit?

Step 2 - If actively exploited (EPSS > 0.5, known exploit). Patch immediately. Emergency deployment. If a patch is not available, apply workaround (WAF rule blocking the exploit pattern, disable the vulnerable feature, network-level mitigation).

Step 3 - If not actively exploited but critical CVSS. Schedule patch within 48 hours. Test the update in staging first. Monitor for exploit attempts (WAF logs, IDS alerts).

Step 4 - If the dependency cannot be updated (breaking changes). Apply compensating controls: WAF rules, input validation, network segmentation to reduce exposure. Schedule the breaking update for the next sprint.

Step 5 - Verify the fix. Run the vulnerability scanner after patching. Verify the CVE no longer appears. Run DAST against the affected endpoint to confirm the vulnerability is not exploitable.

Q74 Kubernetes RBAC misconfigured - Pod has cluster-admin

Step-by-Step

Step 1 - Identify the binding. `kubectl get clusterrolebindings -o json | jq '.items[] | select(.roleRef.name=="cluster-admin")'` shows all entities with cluster-admin. Look for ServiceAccounts that should not have it.

Step 2 - Assess if it was exploited. Check audit logs for API calls from the ServiceAccount. If the Pod was compromised, the attacker had full cluster control - they could create Pods, read all Secrets, modify any resource.

Step 3 - Remove the binding immediately. `kubectl delete clusterrolebinding the-offending-binding`. Create a scoped Role with only the permissions the Pod actually needs.

Step 4 - Rotate all secrets in the cluster if there is any indication of compromise. An attacker with cluster-admin could have read every Secret in every namespace.

Step 5 - Prevent. Use OPA/Kyverno to deny ClusterRoleBindings referencing cluster-admin except from specific admin identities. Audit RBAC quarterly. Use `kubectl auth can-i --list` to verify ServiceAccount permissions match expectations. Automate RBAC auditing with `rbac-police` or `krane`.

Q75 SQL injection in production API

Step-by-Step

Step 1 - Deploy WAF rule immediately. Block requests matching SQL injection patterns on the affected endpoint. This provides immediate protection while you fix the code.

Step 2 - Assess data exposure. What data was the injected query able to access? Check application logs for suspicious queries. Review database audit logs for unauthorized reads or modifications.

Step 3 - Fix the code. Replace string concatenation with parameterized queries (prepared statements). This is the definitive fix - no amount of input filtering is as reliable as parameterized queries.

Step 4 - Deploy the fix. Emergency deployment through the standard pipeline (do not skip CI/CD).

Step 5 - Verify. Run DAST specifically targeting the affected endpoint. Attempt SQL injection manually to confirm the fix works.

Step 6 - Prevent. Add SAST rules that detect string concatenation in SQL queries. Add DAST to the CI/CD pipeline for staging. Train developers on secure coding (parameterized queries). Add input validation as defense in depth.

Q76 S3 bucket with PII exposed publicly

Step-by-Step

Step 1 - Block public access immediately. Enable Block Public Access on the bucket. This overrides any policy or ACL granting public access.

Step 2 - Determine exposure. What PII was exposed? Check bucket contents. How long was it public? Check CloudTrail for when the policy was changed. Were files accessed? Check S3 access logs for unknown IPs and large downloads.

Step 3 - Legal notification. GDPR: notify DPA within 72 hours if EU resident data was exposed. CCPA: similar requirements. Engage legal and compliance teams immediately.

Step 4 - Root cause. Misconfigured Terraform? Manual console change? Missing Block Public Access at the account level?

Step 5 - Prevent. Enable Block Public Access at the ACCOUNT level (not just bucket). AWS Config rule s3-bucket-public-read-prohibited with auto-remediation. Enable Macie for PII detection. SCP preventing s3:PutBucketPublicAccessBlock to false.

Q77 TLS certificate expired - service down

Step-by-Step

Step 1 - Renew immediately. If using cert-manager with Let's Encrypt: `kubectl describe certificate` to check the status. cert-manager should auto-renew 30 days before expiry. If it failed, check: cert-manager Pod logs, DNS records for validation, rate limits (Let's Encrypt has issuance limits).

Step 2 - If cert-manager cannot renew quickly, generate a self-signed certificate as a temporary fix to restore service. Then investigate why renewal failed.

Step 3 - Root cause. cert-manager not running? DNS validation record deleted? ACME account rate-limited? Let's Encrypt outage?

Step 4 - Prevent. Monitor certificate expiry with Prometheus (x509_cert_not_after metric from Blackbox Exporter). Alert 30 days before expiry. Test cert-manager renewal in staging regularly. Use multiple certificate issuers as fallback.

Q78 Insider threat - data exfiltration

Step-by-Step

Step 1 - Confirm and contain without alerting the suspect. Verify the exfiltration with audit logs. Do not confront the employee yet - involve HR and legal first. Restrict access if the threat is ongoing, but use a plausible excuse (system maintenance).

Step 2 - Evidence collection. Audit logs: what data was accessed? Database query logs: what was queried? Network logs: what was downloaded and where was it sent? Email/chat logs (if company policy permits monitoring). Create a timeline of all suspicious activity.

Step 3 - Engage HR and legal. This is a legal matter, not just a technical one. Follow your organization's insider threat policy. Law enforcement may need to be involved depending on the data and jurisdiction.

Step 4 - Contain. Revoke all access. Disable accounts. Change shared credentials the insider had access to. Block external transfer tools. Audit all systems the insider had access to for backdoors.

Step 5 - Prevent. DLP tools monitoring data transfers. Audit logging of all sensitive data access. Behavioral analytics (UEBA) detecting anomalous access patterns. Principle of least privilege. Regular access reviews. Exit procedures that immediately revoke access.

Q79 Ransomware encrypted production servers

Step-by-Step

Step 1 - Isolate immediately. Disconnect affected servers from the network. Do NOT shut down - preserve forensic evidence in memory. Prevent lateral movement to other systems.

Step 2 - Do NOT pay the ransom. Payment does not guarantee recovery. It funds criminal operations. It marks you as a target for future attacks. Consult law enforcement and legal counsel.

Step 3 - Assess scope. Which servers are affected? Are backups intact? Are backup systems isolated from the compromised network? (Attackers often target backups first.)

Step 4 - Recover. Rebuild servers from known-good images. Restore data from verified, uncompromised backups. Rebuild in a clean environment before connecting to the network.

Step 5 - Determine entry vector. Phishing email? Unpatched vulnerability? Compromised credentials? RDP exposed to the internet?

Step 6 - Prevent. Immutable backups (S3 Object Lock, air-gapped backups). Patch management with SLA for critical patches. Network segmentation preventing lateral movement. EDR (Endpoint Detection and Response) on all servers. MFA everywhere. Disable RDP on internet-facing servers. Security awareness training.

Q80 Supply chain attack - compromised npm package

Step-by-Step

Step 1 - Identify affected systems. Which services use this package? Check package-lock.json across all repos. Which versions are affected? Which environments have deployed the compromised version?

Step 2 - Contain. Block the compromised package version in your private registry. Roll back deployed services to the last known-good version (before the compromised package was included).

Step 3 - Assess damage. What does the malicious code do? Common payloads: exfiltrate environment variables (containing secrets), install backdoors, crypto mining, data theft. Review network logs for unexpected outbound connections from affected services.

Step 4 - Rotate all credentials that were accessible to the affected services. The malicious package likely exfiltrated environment variables containing secrets.

Step 5 - Prevent. Use a private npm registry (Artifactory, npm private) as a proxy - scan packages before they reach developers. Pin dependency versions with lock files. Review and audit new dependencies before adding them. Use npm audit and Snyk in CI. Enable package provenance verification. Monitor for typosquatting (similar package names).

Q81 DDoS attack overwhelming application

Step-by-Step

Step 1 - Enable DDoS protection. Activate AWS Shield Advanced or GCP Cloud Armor. Enable auto-scaling to absorb increased traffic while mitigation takes effect.

Step 2 - Identify attack type. L3/L4 (volumetric - UDP flood, SYN flood): Shield/Cloud Armor handles automatically. L7 (application layer - HTTP flood): requires WAF rules to distinguish attack traffic from legitimate traffic.

Step 3 - WAF rules. Rate-limit by IP. Block known-bad User-Agent strings. Implement CAPTCHA challenges for suspected bot traffic. Geo-block regions where you have no customers (if applicable).

Step 4 - CDN as shield. CloudFront/Cloud CDN absorbs traffic at the edge. The origin only receives cache misses - a fraction of total traffic.

Step 5 - Post-attack. Review logs to understand the attack pattern. Update WAF rules to block similar future attacks. Implement traffic fingerprinting. Consider DDoS response retainer (AWS Shield Advanced includes DDoS Response Team).

Q82 Secrets in Docker image layers

Step-by-Step

Step 1 - Identify what was exposed. `docker history image:tag` shows all layers. `docker save image | tar xf -` to extract layers and search for secrets.

Step 2 - Rotate immediately. Every secret found in the image is compromised. Change passwords, regenerate API keys, rotate certificates.

Step 3 - Remove the image from the registry. Anyone who pulled it has the secrets.

Step 4 - Rebuild without secrets. Use BuildKit secret mounts: `RUN --mount=type=secret,id=mykey cat /run/secrets/mykey`. The secret is available during the build step but never written to any layer. Never use ARG or COPY for secrets.

Step 5 - Scan for secrets in CI. Trivy and Gype can detect embedded secrets in images. Add scanning to every build pipeline.

Q83 SSRF accessing cloud metadata

What Happened

An application has a URL parameter that fetches remote content (a preview feature, webhook, URL validator). An attacker submits `http://169.254.169.254/latest/meta-data/iam/security-credentials/` and receives the instance's IAM role credentials.

Step-by-Step

Step 1 - Enforce IMDSv2. IMDSv2 requires a session token obtained via PUT request with a TTL header. SSRF attacks using GET requests fail because they cannot include the required PUT step. Set `HttpTokens: required` in instance metadata options.

Step 2 - Fix the application. Whitelist allowed URL schemes and domains. Block requests to internal IP ranges (169.254.0.0/16, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16). Use a URL parser to validate the resolved IP, not just the hostname (prevent DNS rebinding).

Step 3 - Assess damage. If credentials were stolen, the attacker had the instance role's permissions. Check CloudTrail for API calls from those credentials. Rotate the role's permissions if compromised.

Prevention

IMDSv2 is the single most effective SSRF mitigation on AWS. Enable it on every instance. Apply via organization-wide SCP requiring `HttpTokens: required`. Additionally: validate and restrict URLs in application code, use WAF rules to block metadata IP access, and segment workloads needing URL fetching into isolated environments with minimal IAM permissions.

Q84 K8s admission webhook bypassed - unsigned images

Step-by-Step

Step 1 - Find unsigned images running. Query all Pods for images not from the approved registry: `kubectl get pods -A -o jsonpath='{range .items[*]}.{metadata.namespace}"\t"}.spec.containers[*].image}{"\n"}{end}' | grep -v approved-registry.`

Step 2 - Why was the webhook bypassed? failurePolicy: Ignore - if the webhook was down, Pods were admitted without validation. The webhook only matches specific namespaces - new namespaces were not covered. The webhook crashed and was not monitored.

Step 3 - Fix. Set failurePolicy: Fail for critical webhooks (with appropriate monitoring). Set namespaceSelector to match all non-system namespaces. Add monitoring for webhook health. Restart/fix the webhook service.

Step 4 - Clean up. Delete or replace unsigned images in the cluster. Re-scan all running images with Trivy.

Q85 Log4Shell (CVE-2021-44228) - detect, patch, verify

Step-by-Step

Step 1 - Identify affected applications. Scan all Java applications for log4j 2.x (versions 2.0-beta9 through 2.14.1). Check: dependency trees (mvn dependency:tree, gradle dependencies), container image layers (Trivy, Grype), and fat JARs (search for log4j-core*.jar).

Step 2 - Immediate mitigation. Set the JVM flag `-Dlog4j2.formatMsgNoLookups=true` (disables the vulnerable JNDI lookup). Or set environment variable `LOG4J_FORMAT_MSG_NO_LOOKUPS=true`. This is a temporary fix - not a replacement for patching.

Step 3 - Patch. Upgrade log4j to 2.17.1+ (the definitive fix). Rebuild and deploy all affected applications.

Step 4 - Check for exploitation. Search logs for: `${jndi:ldap://}`, `${jndi:rmi://}`, or any `${jndi:}` pattern. Check network logs for outbound LDAP/RMI connections to unknown IPs. Check for reverse shells or unauthorized processes on affected servers.

Step 5 - Verify. Run a DAST scan specifically testing for Log4Shell. Attempt the exploit in staging to confirm the fix works.

Q86 CI/CD pipeline compromised

Step-by-Step

Step 1 - Isolate the pipeline. Disable the compromised workflow. Remove the runner/agent from the pool.

Step 2 - Assess what was compromised. What secrets did the pipeline have access to? What artifacts did it produce during the compromise window? What was deployed? Check: cloud credentials, registry credentials, deployment targets.

Step 3 - Verify deployed artifacts. If using image signing, verify signatures on all recently deployed images. If not signed, rebuild and redeploy from a clean pipeline.

Step 4 - Rotate all pipeline secrets. Every credential accessible to the compromised pipeline is compromised.

Step 5 - Root cause. Compromised third-party action? Malicious PR modifying the workflow? Compromised runner with persistent access? Stolen pipeline token?

Step 6 - Prevent. Pin actions by SHA. Use OIDC instead of stored credentials. Ephemeral runners. Sign all artifacts. Monitor runner integrity. Review workflow changes in PRs.

Q87 XSS in customer-facing application

Step-by-Step

Step 1 - Deploy CSP. Content-Security-Policy header that blocks inline scripts: `script-src 'self'`. This mitigates most XSS even before the code fix.

Step 2 - Fix the code. Encode all user input before rendering in HTML. Use framework-provided escaping (React's JSX auto-escapes, Django's template auto-escaping). Never use `dangerouslySetInnerHTML` (React) or `|safe` (Django) with user input.

Step 3 - Set HttpOnly on session cookies. Even if XSS executes, it cannot steal session cookies marked `HttpOnly` - JavaScript cannot access them.

Step 4 - Verify. Run DAST targeting the affected endpoint. Attempt XSS payloads manually. Run browser-based testing with security assertions.

Q88 Database credentials hardcoded in source code

Step-by-Step

Step 1 - Rotate the credentials immediately. Change the database password. Update the application to read from a secret manager (Vault, Secrets Manager) instead of the code.

Step 2 - Remove from Git history. `git filter-repo` to remove the file/line containing credentials from all history. Force push.

Step 3 - Check for unauthorized access. Review database audit logs for access from unexpected sources during the exposure period.

Step 4 - Prevent. Pre-commit hooks (`git-secrets`, `gitleaks`) that block commits containing credential patterns. SAST rules detecting hardcoded secrets. CI checks scanning for credential patterns. Developer training on secret management practices.

Q89 500+ critical findings - prioritization

Step-by-Step

Step 1 - Do not try to fix everything at once. Triage into categories: actively exploited (fix now), internet-facing with high EPSS (fix this week), internal with high CVSS (fix this sprint), everything else (schedule).

Step 2 - Apply the formula. $\text{Priority} = \text{CVSS score} \times \text{EPSS probability} \times \text{exposure factor}$ (internet-facing = 3x, internal = 1x) $\times \text{reachability}$ (reachable = 2x, unreachable = 0.5x).

Step 3 - Quick wins. Some findings are fixable in minutes: update a base image (fixes 50+ OS-level CVEs), add a security header (fixes a whole category), enable encryption (fixes multiple compliance findings).

Step 4 - Track progress. Dashboard showing: total findings by severity, age of findings, remediation rate. Report weekly to leadership. Target: zero critical older than 48 hours, zero high older than 7 days.

Q90 Design a complete DevSecOps pipeline

The Architecture

Pre-commit: `git-secrets` + `gitleaks` (secret detection). pre-commit hooks for linting.

PR / Build: SAST (Semgrep/CodeQL - code vulnerabilities). SCA (Snyk - dependency vulnerabilities). License scanning (FOSSA - GPL detection). IaC scanning (checkov - infrastructure misconfigurations). Unit tests with security test cases.

Post-build: Container image build. Image scanning (Trivy - OS/library CVEs). Image signing (cosign - provenance). Push to private registry only if all scans pass.

Pre-deploy: Admission webhooks verify image signature. OPA/Kyverno enforce Pod security policies. DAST against staging (OWASP ZAP). Approval gate for production.

Post-deploy: Runtime security (Falco - detect anomalous behavior). DAST against production (scheduled). Continuous vulnerability scanning of running images. Synthetic monitoring for security-relevant endpoints.

Operations: SIEM integration for security events. Incident response automation (EventBridge → Lambda). Compliance dashboard (Config/Security Hub). Vulnerability management tracking. Security metrics reporting (MTTR, escape rate, finding age).

Q91 JWT token stolen - session hijacking

Step-by-Step

Step 1 - The challenge: JWTs are stateless. The server cannot "revoke" a JWT because it does not track issued tokens. Until the token expires (exp claim), it is valid.

Step 2 - Immediate mitigation. Add a token blocklist (Redis): when a stolen token is identified, add it to a blocklist checked on every request. This defeats the purpose of stateless JWTs but is necessary for incident response.

Step 3 - Force re-authentication. Change the signing key - all existing tokens become invalid. All users must re-authenticate. Disruptive but effective for a major compromise.

Step 4 - Long-term fix. Use short-lived access tokens (15 minutes) with refresh tokens (stored in httpOnly cookies). The access token expires quickly, limiting the window of a stolen token. The refresh token is stored securely and can be revoked server-side. Implement token binding - tie tokens to the client's TLS session or IP address.

Q92 K8s ServiceAccount token auto-mounted

The Problem

By default, Kubernetes mounts a ServiceAccount token into every Pod at `/var/run/secrets/kubernetes.io/serviceaccount/token`. This token authenticates to the API server. If the Pod is compromised, the attacker has API access with whatever permissions the ServiceAccount has.

Fix

Set `automountServiceAccountToken: false` on Pods that do not need API access (most Pods). **Create dedicated ServiceAccounts** with minimal RBAC for Pods that do need API access. **Set `automountServiceAccountToken: false`** on the default ServiceAccount in every namespace. Enforce with Kyverno/OPA: deny Pods that mount the default ServiceAccount token without explicit opt-in.

Q93 Cryptojacking - EC2 mining crypto

Step-by-Step

Step 1 - Detection. GuardDuty finding: `CryptoCurrency:EC2/BitcoinTool.B!DNS` (DNS queries to crypto mining pools). Or: unexpected CPU utilization at 100% for days. Or: AWS bill spike from compute costs.

Step 2 - Isolate. Change the instance's SG to deny all outbound traffic (stops the mining). Do not terminate - preserve evidence.

Step 3 - Investigate. How did the attacker get in? Check: exposed SSH keys, unpatched vulnerabilities, compromised web application, stolen AWS credentials, open Docker socket.

Step 4 - Capture and terminate. AMI snapshot for forensics. Terminate the instance. Launch a clean replacement.

Step 5 - Prevent. GuardDuty in all regions and accounts. Close unnecessary ports. Patch regularly. Use IMDSv2. Restrict outbound traffic with SGs and NACLs. Monitor for DNS queries to known mining pool domains.

Q94 Dependency confusion - internal package hijacked

What Happened

Your company has an internal npm package called @company/auth-utils published on a private registry. An attacker publishes auth-utils (without the scope) on the public npm registry with a higher version number. Your CI pipeline resolves the public package (higher version) instead of the internal one. The public package contains malicious code that exfiltrates environment variables.

Step-by-Step

Step 1 - Block the public package. Configure npm to ONLY resolve @company/* packages from your private registry (.npmrc: @company:registry=https://private-registry.example.com/). For unscoped packages, use the private registry as a proxy that blocks packages with the same name as internal packages.

Step 2 - Check if you were affected. Review CI build logs for which version of the package was installed. Check package-lock.json for unexpected registry URLs. If the public package was installed, rotate all credentials accessible to the build.

Step 3 - Prevent. Always use scoped packages (@company/package-name). Configure registry resolution in .npmrc to prevent fallback to public registries. Claim your internal package names on the public registry (publish empty placeholder packages). Use package-lock.json with integrity hashes.

Q95 mTLS certificate expired - service mesh down

Step-by-Step

Step 1 - Confirm. Check service mesh logs. Istio: istioctl proxy-status shows which sidecars have expired certificates. Linkerd: linkerd check shows certificate status.

Step 2 - Root cause. The mesh CA certificate expired. Istio's self-signed CA has a default 10-year root cert but the intermediate has a shorter lifetime. If cert-manager or Vault manages the CA, check their renewal status.

Step 3 - Fix. For Istio: restart istiod to generate new certificates. If the root CA expired, you need to rotate it - a more complex procedure requiring rolling restart of all sidecars. For Linkerd: linkerd upgrade to rotate trust anchors.

Step 4 - Verify. All inter-service traffic should resume. Check istioctl proxy-status or linkerd check for certificate validity.

Step 5 - Prevent. Monitor certificate expiration dates (x509_cert_not_after metric). Alert 30 days before any certificate expires - CA, intermediate, and leaf certificates. Automate CA rotation. Test rotation procedures in staging quarterly.

Q96 WAF blocking legitimate traffic

Step-by-Step

Step 1 - Identify blocked requests. Check WAF logs for requests matching the blocked rule. Compare blocked request patterns with legitimate traffic.

Step 2 - Create an exclusion. Exclude the specific path, parameter, or request pattern that is generating false positives. Do not disable the entire rule - scope the exclusion narrowly.

Step 3 - Switch to count mode temporarily. If the false positive rate is high, switch the problematic rule to count mode (log but do not block) while you tune.

Step 4 - Root cause. Common causes: a form field containing HTML (the WAF thinks it is XSS), a JSON API body containing SQL keywords (the WAF thinks it is SQLi), a large POST body exceeding the WAF's inspection limit.

The Tension

Security wants to block threats. Business wants zero false positives. The WAF engineer's job is tuning the rules to maximize true positives while minimizing false positives. This is an ongoing process, not a one-time configuration.

Q97 Terraform state with secrets in S3

The Problem

Terraform state files contain sensitive data in plaintext: database passwords, API keys, certificates. If the S3 bucket storing the state is misconfigured (public, wrong permissions), all secrets are exposed.

Step-by-Step

Step 1 - Rotate all secrets referenced in the state file. Database passwords, API keys, certificate private keys - everything.

Step 2 - Secure the bucket. Enable S3 Block Public Access. Enable SSE-KMS encryption. Enable versioning. Enable S3 access logging. Restrict bucket policy to specific IAM roles. Enable MFA delete.

Step 3 - Lock the state. Use DynamoDB state locking to prevent concurrent modifications. Use a dedicated S3 bucket for state (not shared with other data).

Step 4 - Prevent secrets in state. Use sensitive = true on Terraform variables (prevents display in CLI output but does NOT prevent storage in state). The real fix: use external secret references (Vault data sources) instead of storing secrets as Terraform resources. Mark sensitive outputs.

Q98 Developer laptop stolen with production access

Step-by-Step

Step 1 - Revoke all access immediately. Disable the developer's SSO/IAM account. Revoke all active sessions. Deactivate access keys. Remove SSH keys from servers. Revoke VPN certificates.

Step 2 - Change shared credentials. Any credentials the developer had access to (even if stored in a password manager on the laptop) should be rotated. Shared service accounts, API keys, database passwords.

Step 3 - Check for unauthorized access. Review audit logs from the time of theft. Check for access from unusual locations or times. Monitor for the next 30 days.

Step 4 - Remote wipe. If MDM (Mobile Device Management) is installed, trigger a remote wipe. If full-disk encryption (FileVault, BitLocker, LUKS) is enabled, the data is protected even without a wipe - the attacker cannot read the disk without the encryption password.

Prevention

Full-disk encryption mandatory on all devices. MDM with remote wipe capability. SSO for all services (revoke one account, revoke all access). MFA on everything. Short-lived credentials (OIDC, session timeouts). No long-lived tokens stored locally. VPN with certificate-based auth (revocable). Consider hardware security keys that never leave the device - if the laptop is stolen, the key is still with the user.

Q99 DNS hijacking - traffic redirected

Step-by-Step

Step 1 - Confirm. dig yourdomain.com shows unexpected IP addresses. Compare with your known infrastructure IPs. nslookup from multiple locations to confirm it is not a local issue.

Step 2 - Check your DNS provider. Was your DNS account compromised? Check for unauthorized record changes. Check audit logs for who made changes and when.

Step 3 - Fix the records. Restore correct DNS records. Change DNS provider credentials. Enable MFA on the DNS provider account.

Step 4 - Assess impact. During the hijack period, traffic went to the attacker's server. If HTTPS was used AND users verified the certificate, their data was safe (the attacker cannot get a valid certificate for your domain without control of the domain). If HTTP was used, all traffic was intercepted - passwords, tokens, data.

Step 5 - Prevent. DNSSEC (prevents record spoofing). Registry lock on your domain (prevents unauthorized transfer). MFA on registrar and DNS provider accounts. Monitor DNS records with external tools (detect unauthorized changes). Use CAA records to restrict which CAs can issue certificates for your domain.

Q100 Privileged container in production - runtime detection

Step-by-Step

Step 1 - Detect. Falco alert: "Privileged container started" or "Container running with sensitive mount." Or: `kubectl get pods -A -o json | jq '.items[] | select(.spec.containers[].securityContext.privileged==true) | .metadata.name'`.

Step 2 - Assess. Is this container legitimate (some system components need privileged)? If unexpected: who created it? When? Check Kubernetes audit logs for the kubectl apply or Helm install that created it.

Step 3 - If unauthorized. The container has full host access. Treat as a potential container escape. Isolate the node. Capture evidence. Check for: processes running on the host, new files in host filesystem, unauthorized network connections.

Step 4 - Remove. Delete the privileged Pod/Deployment. Cordon and drain the node. Replace the node (cannot trust the host after privileged container access).

Step 5 - Prevent. Pod Security Admission Restricted profile (blocks privileged containers). Kyverno/OPA policy denying privileged: true. Falco runtime alert for privileged container creation. Only allow privileged containers in kube-system namespace for system components.

Interview Tips

For Junior Roles

Focus on Q1-Q20. Know CIA triad, OWASP Top 10, encryption basics, and secret management principles. Be able to explain the difference between authentication and authorization. Understand where security fits in a CI/CD pipeline.

For Mid-Level Roles

Q1-Q50. Be ready to discuss SAST/DAST/SCA integration, container hardening, Kubernetes security contexts, JWT security, and threat modeling. Know how to set up image scanning and signing in a pipeline. Have a real story about a security incident you handled.

For Senior Roles

All 100 questions. Zero trust architecture, mTLS/SPIFFE, policy as code, runtime security (Falco), and compliance automation separate senior from mid-level. Be ready to design a complete DevSecOps pipeline and discuss security architecture trade-offs.

For Scenario Questions

Walk through the incident response framework: detect → contain → investigate → remediate → prevent. Every scenario answer must end with prevention - the interviewer wants to see that you think about long-term security posture, not just firefighting.

Quick Reference Cheat Sheet

Topic	Key Takeaway
DevSecOps	Shift left AND right. Security in every phase. Automate everything. Culture matters most.
CIA triad	Confidentiality, Integrity, Availability. Every decision balances all three.
OAuth/OIDC	OAuth = authorization. OIDC = authentication on top of OAuth. Use PKCE for mobile/SPA.
TLS/mTLS	TLS encrypts. mTLS verifies both sides. Use mTLS for service-to-service.
OWASP Top 10	Injection, broken auth, misconfiguration are the top three. Know all ten.
Secrets	Vault/Secrets Manager. Never in code. Never in env vars if avoidable. Dynamic/OIDC is best.
SAST/DAST/SCA	SAST on every PR. SCA on every PR. DAST on staging nightly. All three are needed.
Container hardening	runAsNonRoot, readOnlyRootFilesystem, drop ALL caps, no privilegeEscalation.
K8s security	PSA Restricted. Dedicated ServiceAccounts. NetworkPolicies default-deny. Encrypt secrets at rest.
Image signing	cosign in CI. Admission webhook verifies. Block unsigned images.
Supply chain	SLSA Level 3. Pin dependencies by hash. Private registry. SBOM for every artifact.
JWT	Short-lived (15 min). RS256 over HS256. HttpOnly cookies. Never alg:none.
Zero trust	Never trust network location. Verify identity + device + context on every request.
Falco	Runtime eBPF detection. Catches container escapes, shells, sensitive file access.
Policy as code	OPA for multi-platform. Kyverno for K8s-only (simpler). Enforce, don't just audit.
Incident response	Detect → Contain → Investigate → Eradicate → Recover → Lessons learned.
Compliance	Automate with Config/Policy. Continuous, not annual. Compliance as code.

Vulnerability priority	CVSS × EPSS × exposure × reachability. Not all criticals are equal.
Credential leak	Rotate immediately → audit CloudTrail → revoke sessions → clean Git history → prevent with OIDC.
WAF	Start in log mode. Tune false positives. Then enforce. Ongoing process.